

# **From Browsing Signal to Promotion Decision**

*A Complete ML System for Co-Mart E-commerce*

*University of Cincinnati*

*BANA 7075*

*Professor Brad Boehmke*

*Group 2*

Oladele Awonusi, Shawn Brown, Judith Geraci, Kristin Jones, Hoda Nassar

April 22, 2026

## Table of Contents

|                                                    |    |
|----------------------------------------------------|----|
| Executive Summary .....                            | 3  |
| Problem Definition.....                            | 3  |
| Technical Design (System Architecture).....        | 4  |
| Data Pipeline and Data Engineering (DataOps) ..... | 5  |
| Model Performance (ModelOPS) .....                 | 5  |
| <i>Model Comparison</i> .....                      | 5  |
| <i>Threshold Tuning</i> .....                      | 6  |
| Deployment Strategy (DevOps).....                  | 7  |
| <i>Monitoring</i> .....                            | 8  |
| <i>Class Imbalance (84.5% non-purchase)</i> .....  | 9  |
| <i>Promotion Targeting Range</i> .....             | 9  |
| <i>Metric Selection Trade-offs</i> .....           | 9  |
| <i>Data Limitations</i> .....                      | 9  |
| Future Improvements .....                          | 10 |
| Conclusion .....                                   | 10 |
| GitHub Repository Link & Dataset Link.....         | 11 |

# Executive Summary

This project presents a machine learning system to support targeted promotion decisions in e-commerce. Using a business scenario of Co-Mart, an online clothing retailer with low session purchase-conversion rate, the aim was to identify sessions that are likely to end without a purchase and identify which of those sessions would be strong candidates for promotional intervention. Multiple classification models, including Logistic Regression, Decision Tree, Random Forest and XGBoost, were evaluated using performance metrics such as ROC-AUC and precision. A champion-challenger approach was then used to compare the top models, and XGBoost was selected as the final champion model. Additional threshold tuning was also performed to balance promotion targeting with wasted discount rate, and a 20%-50% intervention range was selected as the final strategy. The final prototype supports batch scoring and individual session scoring and provides a flexible interface that allows threshold settings to be adjusted as business needs evolve.

## Problem Definition

In the world of e-commerce, companies invest heavily in attracting visitors to their website, yet only a small portion of those visits result in a purchase. In our project, we frame this problem around Co-Mart, an online clothing retailer. Co-Mart is facing an issue where only 15.5% of browsing sessions end in a purchase, meaning that most visitors leave the site without buying anything. To address the issue, Co-Mart is considering offering a 15% discount to customers to increase purchase rate. However, the company wants to ensure that the offer is targeted to the right customers rather than broadly offered to everyone, which could lead to reduced margins and wasted discounts. Therefore, the business challenge here is not simply increasing traffic, but identifying which sessions are more likely to end without a purchase and are strong candidates for the targeted promotion. Our goal is to support more efficient promotion decisions by helping the business focus promotion interventions where they are more likely to have an impact.

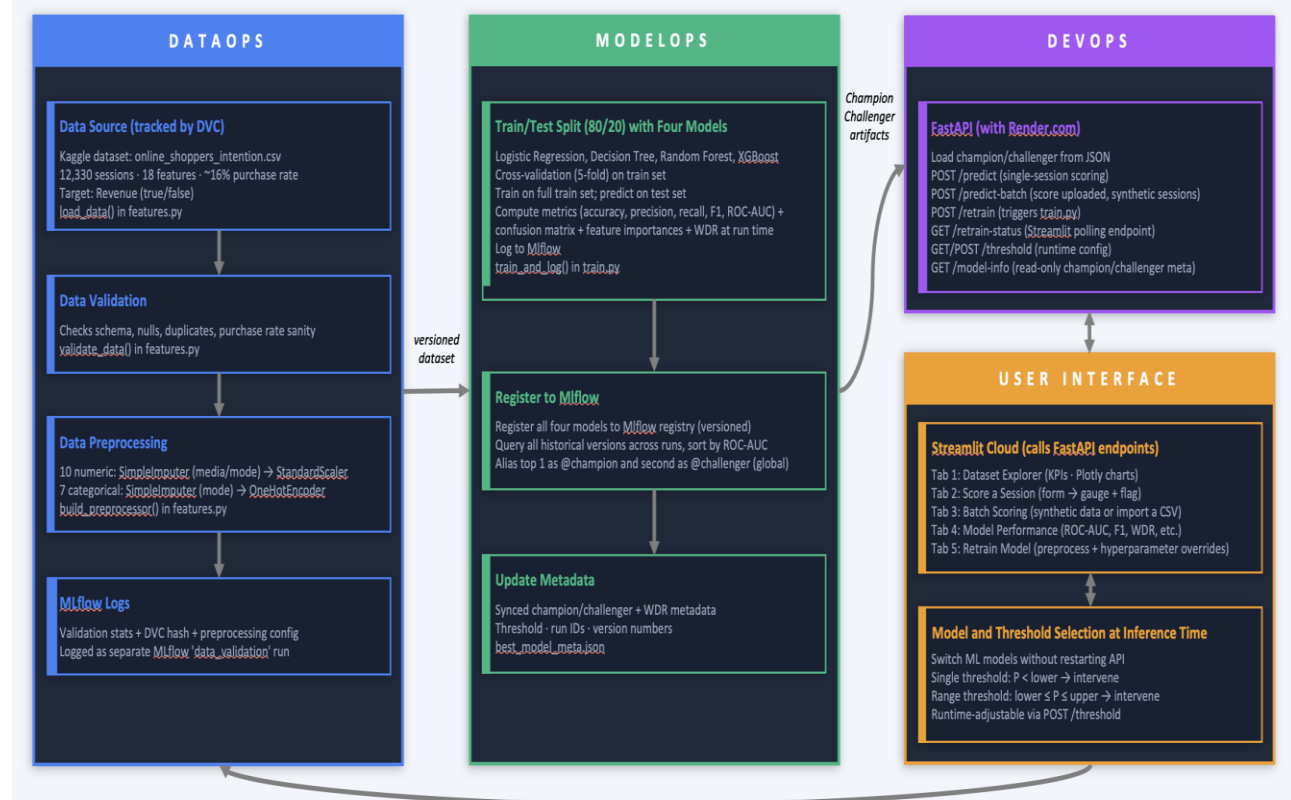
A machine learning solution is appropriate for this problem because customer purchase behavior depends on many interacting signals that are difficult to capture with simple if-then rules. Variables such as time spent on pages, browsing activity, visitor type and engagement patterns may together hint whether a session is likely to end with or without a purchase. A rule-based approach would be too rigid and may miss some patterns due to that, while machine learning can use historical data to learn these useful relationships and identify the sessions that are more likely to end without a purchase. In this way, machine learning offers a more data-driven and scalable approach for reducing the number of wasted discounts.

# Technical Design (System Architecture)

Our system is built on a modular MLOps architecture that transitions the Online Shoppers Intention dataset into a production-ready FastAPI service. By decoupling the system into 4 distinct operational tiers, we ensure that session simulation, feature engineering, and model scoring remain independent, yet synchronized. The DataOps tier manages the lifecycle of 12,330 sessions from a versioned UCI dataset obtained from Kaggle (<https://www.kaggle.com/datasets/henrysue/online-shoppers-intention>) tracked by DVC through to a shared feature engineering pipeline (features.py) for consistency across model inputs. Using MLflow via DagHub, the ModelOps tier manages experimentation and model selection, prioritizing ROC-AUC to promote an XGBoost “Champion” from a competition of 4 models. The Decision Layer tier takes the model’s prediction and applies a runtime-adjustable business threshold (between 20% and 50%) to determine if a session should be flagged for a promotion. The final tier, the User Interface tier, consists of a Streamlit dashboard hosted on Streamlit Cloud, that provides stakeholders with dataset exploration, batch scoring, and the ability to trigger a retraining loop via the FastAPI backend.

**Figure 1** illustrates the flow from our DataOps versioning through to the ModelOps registry and the Decision Layer logic, all surfaced via the User Interface.

**Figure 1: ML System Architecture**



## Data Pipeline and Data Engineering (DataOps)

The data foundation is the Kaggle Online Shoppers Intention dataset, comprising 12,330 session-level records across 18 behavioral and time-based features, with a purchase rate of approximately 15.5%. Raw data is version-controlled using DVC, and its MD5 hash is logged to every MLflow training run, ensuring that any model can be traced back to the exact data snapshot that produced it. Before training begins, an automated validation layer — implemented in `validate_data()` within `features.py` — enforces schema integrity by confirming all expected columns are present, checks that the target variable contains only binary values, flags duplicate rows, and applies range validation on features such as `BounceRates` and `ExitRates`, which are expected to fall within 0 to 1. If any critical check fails, training is aborted and the error is logged; non-blocking issues such as elevated null rates are recorded as warnings in a dedicated MLflow `data_validation` run alongside descriptive statistics including row count, purchase rate, and per-column null percentages. Feature transformation is handled by `build_preprocessor()`, also in `features.py`, which constructs a `ColumnTransformer` that applies median imputation followed by standard scaling to the 10 numeric features, and mode imputation followed by one-hot encoding to the 7 categorical features. Label leakage is prevented by design: every feature in the model (page view counts, session durations, bounce and exit rates, page values, month, visitor type, and weekend flag) reflects behavior observable during the browsing session itself, prior to any purchase decision. The critical architectural choice is that both `train.py` and `api/main.py` import `build_preprocessor()` from the same `features.py` module, making training-serving skew architecturally impossible: there is a single source of truth for all feature transformations across the entire system.

## Model Performance (ModelOPS)

### *Model Comparison*

To clearly identify whether a session will result in a purchase or not, different classification models were compared, including Logistic Regression, Decision Tree, Random Forest and XGBoost. These models were evaluated using ROC-AUC and precision as performance metrics (**Table 1**). ROC-AUC was used to measure each model's overall ability to differentiate between purchase and non-purchase sessions, while precision was especially important because it reflects how accurately the model identifies sessions for possible intervention. In this project, higher precision is desirable because it helps reduce the number of unnecessary promotions and therefore lower wasted discounts.

After the initial model comparison, we adopted a champion-challenger strategy to compare the highest performing models more directly at a specified threshold setting. Based on the results shown in the table below, XGBoost was selected as the champion model and Random Forest as

the challenger. XGBoost achieved a ROC-AUC of 0.9338 and a precision of 56.96%, while Random Forest achieved a ROC-AUC of 0.9315 and a precision of 55.12%.

**Table 1: Model Comparison**

| Model         | ROC-AUC | Precision |
|---------------|---------|-----------|
| XGBoost       | 0.9338  | 56.96%    |
| Random Forest | 0.9315  | 55.12%    |

### *Threshold Tuning*

After selecting the champion model, XGBoost, we performed additional threshold tuning to determine how the model should be used in real-world settings. Rather than relying on default classification cutoffs, we tested different thresholds to evaluate the trade-off between wasted discount rate, which is the rate of how many sessions were given a discount while they were already going to make a purchase without intervention, and the type of sessions targeted for intervention. Two main strategies were considered: single-threshold mode, in which sessions below a given purchase probability cutoff were flagged for intervention, and range mode, in which only sessions within a selected probability range were targeted.

The threshold comparison showed that the lower single-threshold cutoffs, such as below 20%, produced the lowest wasted discount rate, while broader range modes gave higher wasted discount rates (**Table 2**). However, the goal of the promotion was not to simply minimize the wasted discounts at all costs, but also to focus promotions on sessions that were more likely to be influenced by a discount offer. Given this approach, we selected the 20%-50% range as the final targeting strategy. This range avoids targeting the sessions that are extremely unlikely to purchase as well as those already highly likely to make a purchase without intervention and instead focuses on mid-probability sessions that may be more responsive to an offer. Although this strategy involves a higher wasted discount rate than the more conservative single-threshold ones, it better coordinates the model's output with the project's real business objectives of targeting influenceable sessions.

**Table 2: Threshold Tuning**

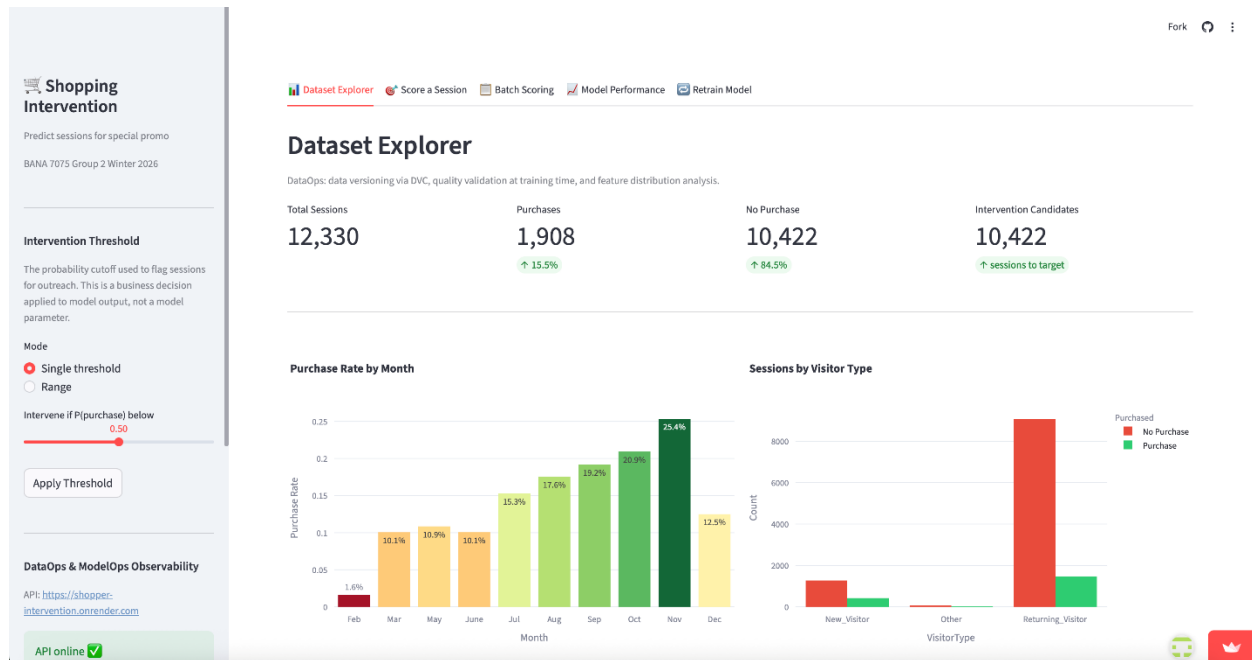
| Threshold Model  | Threshold Level | Wasted Discount Rate |
|------------------|-----------------|----------------------|
| Single-threshold | Below 20%       | 8.3%                 |
| Single-threshold | Below 30%       | 11.6%                |
| Single-threshold | Below 50%       | 17.8%                |
| Range            | 20%-40%         | 28.9%                |
| <b>Range</b>     | <b>20%-50%</b>  | <b>33.6%</b>         |
| Range            | 30%-50%         | 39.9%                |
| Range            | 30%-60%         | 45.0%                |

Although the 20%-50% range was selected for Co-Mart’s current targeting strategy, the prototype interface allows users to change the threshold settings and immediately see how those changes affect the intervention results. This allows the system to be more flexible and to evolve with changing business priorities over time.

## Deployment Strategy (DevOps)

The inference layer is built on FastAPI and deployed to Render.com, exposing a REST API that serves predictions to any downstream system. The primary endpoint, `POST /predict-batch`, accepts a set of session feature vectors and returns purchase probability scores along with intervention flags for each session, supporting the project's batch scoring approach. The intervention decision itself is encoded in a separate `GET/POST /threshold` endpoint, which reads and writes the probability cutoff from `best_model_meta.json` at runtime. This design separates the business rule from the model: stakeholders can adjust the targeting range — for example, switching from the 20–50% range to a more conservative single-threshold cutoff — without retraining or redeploying the API. The Streamlit dashboard (**Figure 2**) hosted on Streamlit Community Cloud, serves as the operational front end: it calls all FastAPI endpoints directly, exposes the batch scoring workflow with downloadable results, and surfaces the threshold configuration panel so non-technical users can modify intervention logic through a form rather than an API call. Both the champion and challenger models are loaded into memory at API startup by resolving their MLflow registry aliases (`@champion`, `@challenger`), so the serving layer always reflects the globally best-performing models without any code change. At inference time, the user can select which model scores for a given session, enabling lightweight A/B comparison in production. Infrastructure is intentionally lean for this project: Render provides the compute for the API on a free-tier web service, and Streamlit Community Cloud hosts the dashboard, keeping operational costs near zero while maintaining a publicly accessible, always-on system.

**Figure 2: Streamlit Dashboard for Dataset Exploration and Intervention Threshold Configuration**



## Monitoring

The system’s long-term viability is maintained through a structured approach to monitoring and feedback. Every training run is logged via MLflow, which tracks ROC-AUC as the primary performance metric while using precision, recall, and the wasted discount rate as critical business guardrails. To address model performance over time and mitigate behavior drift, we integrated a data validation layer that executes multiple quality checks before training to ensure feature distributions remain stable. Threshold tuning is handled as a dynamic business decision rather than a static model parameter. Finally, the architecture “closes the loop” through an Async Retraining pipeline. A technical stakeholder can trigger a new training run directly from the dashboard, which spawns a `train.py` subprocess that pulls the latest versioned data, evaluates new champions, and automatically reloads the updated model into the FastAPI once the run is complete.

To align with production-grade expectations, the monitoring strategy for the Co-Mart system would need to transition from manual oversight to automated detection of data and concept drift. Automated drift detection could be achieved by implementing statistical profiling, such as the Population Stability Index, to continuously compare the distribution of live features like PageValues and ExitRates against the training baseline. This framework would include automated alerting via the FastAPI backend to notify the DataOps team if feature distributions shifted significantly, signaling a change in shopper behavior that might invalidate the current model.



Furthermore, the system could maintain performance and business guardrails by closing the feedback loop through live metric tracking. By logging session outcomes, the system could monitor real-time Precision and ROC-AUC to detect performance degradation. The Wasted Discount Rate serves as a primary business audit; if this metric exceeded the established 33.6% target for the 20%–50% range, it would trigger an automated retraining event. Finally, the champion-challenger architecture would allow the Challenger model to run in "shadow mode," providing a continuous benchmark to determine if evolving user patterns necessitated a model swap to maintain optimal promotion targeting.

## Challenges and How They Were Addressed

These challenges reflect key DataOps and ModelOps considerations within Co-Mart's promotion targeting system, particularly in handling data limitations and aligning model performance with business objectives of reducing wasted discount rate.

### *Class Imbalance (84.5% non-purchase)*

The dataset is heavily imbalanced, making accuracy a misleading metric. This was addressed by deprioritizing accuracy and focusing on ROC-AUC and precision, along with threshold tuning to control false positives.

### *Promotion Targeting Range*

Determining the optimal threshold to balance wasted discounts with targeting customers who are most likely to convert to a purchase when offered a 15% discount. This was addressed by evaluating multiple probability bands and selecting the 20%-50% threshold range, which provided the best balance between maximizing conversion opportunities while minimizing wasted discounts.

### *Metric Selection Trade-offs*

The use of multiple metrics, including ROC-AUC, precision, and F1, creates trade-offs in model selection since different metrics favor different models. In this project, these trade-offs were especially important because the goal for Co-Mart is to minimize wasted discounts while still identifying sessions that may benefit from a promotion. This was addressed by prioritizing metrics aligned with the business objective of increasing revenue without excessive wasted discounts, especially focusing on ROC-AUC, precision, and a wasted discount rate metric to guide model selection.

### *Data Limitations*

The dataset lacks unique session identifiers, making it difficult to confirm true duplicates or identify highly similar sessions. For this project, sessions with identical behavior were treated as separate observations. Additionally, the dataset does not include all months of the year, which

limits the model's ability to capture full seasonal patterns in user behavior and account for the factors such as holiday shopping surges or seasonal clothing trends. These limitations were addressed through consistent data validation and preprocessing to ensure reliable model inputs despite these limitations.

## Future Improvements

The current system uses batch scoring for evaluation while being designed to support real-time deployment in a future phase through a modular architecture. Future iterations should include ongoing monitoring of model performance and periodic retraining as user behavior evolves. While the current version of the model successfully demonstrates predictive accuracy, it operates primarily through batch processing where users upload session data via the Streamlit interface to receive scores. In a live e-commerce environment, the window of opportunity to influence a shopper is narrow; decisions must be made in milliseconds while the user is still actively browsing. To bridge this gap, a key future enhancement will be the full operationalization of the FastAPI /predict endpoint for single-session, real-time requests. This update will move the system beyond file-based uploads to an event-driven model. Instead of waiting for a batch process, the backend will accept individual data points for a single shopper—such as current page duration and bounce rate—and return an immediate "Yes/No" intervention decision. This evolution will allow Co-Mart to deploy the model as a live microservice, enabling truly dynamic promotional triggers that respond to shopper behavior as it happens. Additionally, implementing A/B testing will allow Co-Mart to measure the true impact of the targeted 15% promotion on conversion rates. Further development could incorporate dynamic discounting using reinforcement learning to determine the minimum incentive required to influence a purchase decision.

As the system is used in practice and implemented more broadly, several considerations will become important to Co-Mart's promotion targeting approach. The model may disproportionately target certain groups of users based on browsing behavior patterns, which could lead to unfair distribution of discounts. Additionally, biases present in the historical session data may influence which users are more likely to receive promotions. There is also a risk of over-targeting users with frequent discounts, which could impact customer perception and long-term revenue. To address these concerns, the system design emphasizes transparency in how predictions are used and supports ongoing monitoring to ensure that promotional strategies remain fair and aligned with business objectives.

## Conclusion

This project demonstrates how machine learning transforms e-commerce promotion strategies by shifting from rigid rules to data-driven, behavioral targeting. By integrating a champion XGBoost model with a dynamic 20%–50% intervention range, the system successfully focuses

discounts on influenceable, mid-probability sessions while protecting Co-Mart's margins through decision-oriented metrics like the wasted discount rate. Beyond initial deployment, the modular architecture supports long-term viability by transitioning to automated monitoring for data and concept drift, using statistical profiling and real-time precision audits to ensure the model evolves with changing consumer behavior. Ultimately, this system proves that the true value of machine learning in retail lies in its ability to support flexible, real-time decisions that optimize both conversion opportunities and operational cost efficiency.

## GitHub Repository Link & Dataset Link

Github Repo: [https://github.com/smbrownai/shopper\\_intervention](https://github.com/smbrownai/shopper_intervention)

Streamlit: <https://shopintervene.streamlit.app>

Dataset Link: <https://www.kaggle.com/datasets/henrysue/online-shoppers-intention>